



SVEUČILIŠTE U ZAGREBU
Fakultet elektrotehnike i računarstva

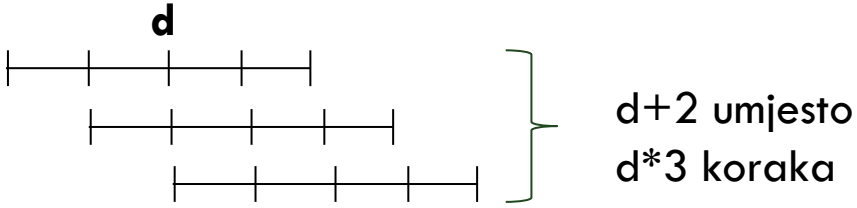
PARALELNO PROGRAMIRANJE

Poglavlje 04 - APRAM



APRAM MODEL

- Asinkroni PRAM – bliže stvarnim računalima
- **Izmjene PRAM modela**
  - procesori su **nesinkronizirani** – nezavisna brzina
  - **GLOBALNA** i **LOKALNA** memorija
    - pristup **lokalnoj memoriji** obavlja se u **jednom**, a pristup **globalnoj memoriji** u *d* koraka
    - uzastopni pristupi globalnoj memoriji mogu se ulančavati!



- većina PRAM algoritama zahtijeva izmjene!
  - PRAM algoritmi nisu nužno optimalni na APRAM-u

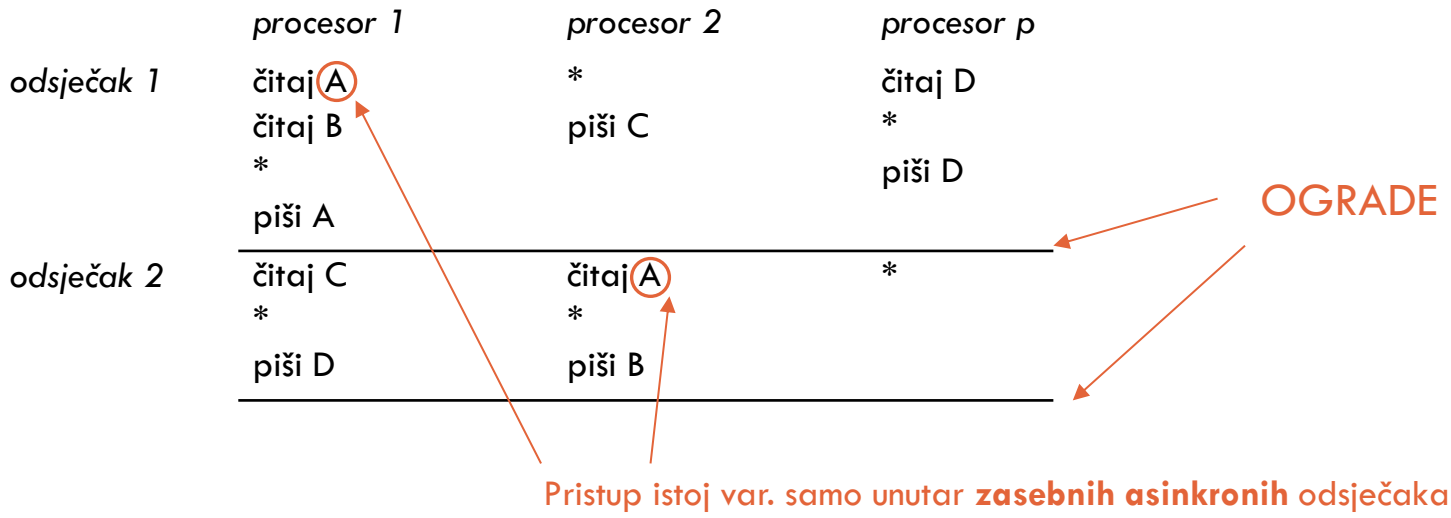
APRAM MODEL

- Nesinkronizirani **MIMD**
  - ***p* procesora** (fiksno!)
  - ***p* lokalnih memorija**
  - **1 globalna memorija**
- Vrste instrukcija
  - **globalno čitanje** – prepisivanje sadržaja globalne u lokalnu memoriju
  - **globalno pisanje**

  } **d koraka**
  - **lokalna operacija**
  - **sinkronizacija: OGRADA** (~ MPI_Barrier)
    - SVI procesori čekaju da svi dođu na isto mjesto

APRAM MODEL

- APRAM program
  - niz asinkronih **odsječaka** odvojenih **ogradama**
  - **samo jedan procesor** smije pristupiti **istoj globalnoj lokaciji** unutar istog asinkronog odsječka!
  - **zadnja instrukcija** (kraj programa) je uvijek **ograda**!



PRIMJER: OTKRIVANJE POČETKA LISTE

- potrebna SINKRONIZACIJA (ograda)!

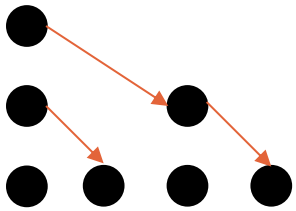
EREW PRAM

```
PAR (i=0 do n-1){
    M[i]=i                                   nužna sinkronizacija za APRAM
    AKO (sljedbenik[i]!=null)
       M[sljedbenik[i]]=-1                   nužna sinkronizacija za APRAM
}
GLAVA = MAX_REDUCE (M[])
```

PRIMJER: RASPODJELA VRIJEDNOSTI

- EREW PRAM → binarno stablo

a) binarno stablo

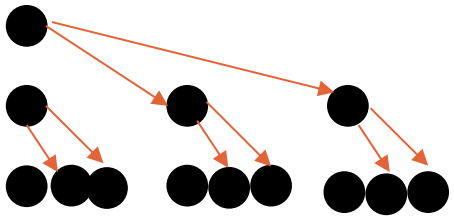


… (x8)

	PRAM	APRAM, d=3
	1	3
	1	3
	1	3

ulančavanje između razina u stablu nije moguće zbog sinkronizacije između razina

b) trinarno stablo



	PRAM	APRAM, d=3
	2	4
	2	4

s obzirom da u jednom koraku svaki piše na 2 mem. lokacije, tu je moguće ulančavanje

OCJENA SLOŽENOSTI

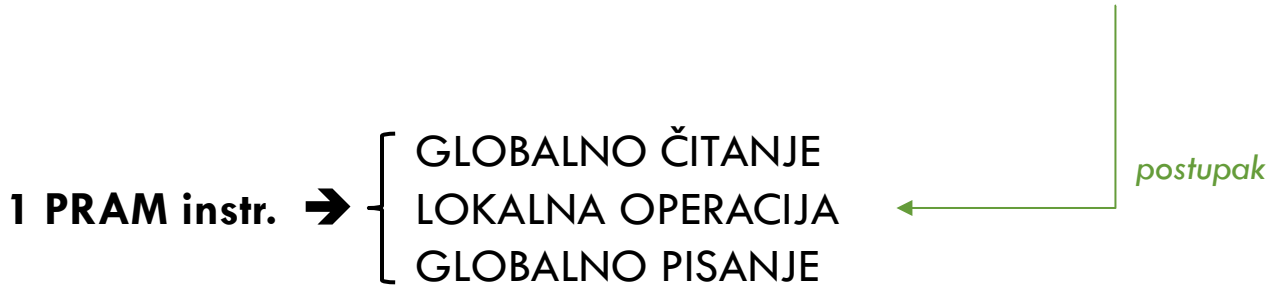
- lokalna operacija – jedinična mjera složenosti
- **globalni pristup: d koraka**
  - uzastopnih k pristupa: **d+k-1 koraka**
- **sinkronizacija: B koraka, B=f(p)**

f-ja broja procesora,
približno monotono rastuća,
npr. $B = \sqrt{p}$

Općenito vrijedi: **2 <= d <= B <= p**

PRILAGODBA PRAM ALGORITAMA

- Postoji već mnogo razvijenih PRAM algoritama te nije isplativo razvijati nove za APRAM, već je bolje prilagoditi postojeće
- Razmatra se **JEDNA PO JEDNA PRAM instrukcija** (automatizacija)



- Moguće **varijante:**
  - **JEDNAK** broj procesora kao i na PRAM-u
  - **MANJE** procesora nego na PRAM-u

PRILAGODBA UZ JEDNAK BROJ PROCESORA

- **jedna EREW PRAM instrukcija:**

  čitanje: d koraka → **GLOB. ČITANJE**

  ograda: B koraka → **da se osigura da su svi pročitali**

  lokalna instr.

  pisanje: d → **GLOB. PISANJE**

  ograda: B → **da se osigura da su svi zapisali rezultat**

→ ukupno: **2B + 2d + 1 → korak: O(B)** → DODATNA SLOŽENOST koja se MNOŽI sa složenošću PRAM-a (npr. O(B)·O(log n))

npr: p = 1024, B = $\sqrt{p}$, d = 3; **K = 71**

↓

(1 korak PRAM → 71 korak APRAM)

PRILAGODBA UZ MANJE PROCESORA

- manje procesora → manji trošak sinkronizacije
  - **n** – br. PRAM procesora
  - **p** – br. APRAM procesora

  → 1 APRAM processor simulira $^{n}/_{p}$ PRAM procesora
- jedna EREW PRAM instrukcija:
  - **glob. čitanje** za **n/p** procesora: **n/p + d-1 koraka** (*ulančavanje!*)
  - **ograda**: B koraka
  - **lokalne instr.**: n/p
  - **glob. pisanje**: n/p + d-1
  - **ograda**: B

trošak sinkr. može biti i neka druga f-ja

PRIMJER: PRAM n=1024, APRAM p=256

- $B = \sqrt{p}$=16, d=3
- 1 PRAM instr.:

$$\mathbf{2} \cdot \underbrace{\left(\frac{\mathbf{1024}}{\mathbf{256}} + \mathbf{3} - \mathbf{1}\right)}_{\text{glob. R/W}} + \underbrace{\mathbf{2} * \mathbf{16}}_{\text{B}} + \underbrace{\frac{\mathbf{1024}}{\mathbf{256}}}_{\text{lok. instr.}} = \mathbf{48}$$

- složenost: $\boldsymbol{O}(\boldsymbol{B} + \frac{\boldsymbol{n}}{\boldsymbol{p}}) \cdot O(t)$

APRAM REDUCIRANJE

- Primjer: n = 1024, B = $\sqrt{p}$ = 32, d = 3

  binarno stablo nije optimalno za APRAM

  a) Algoritam uz **p = n** procesora ➔ **B-arno stablo, dubina (log n / log B)**

     - **jedna razina stabla:**

       B-1 čitanje (d+B-2) 1 processor kombinira svoju vrijednost s još njih B-1

       zbrajanje (B-1)

       pisanje zbroja (d)

       ograda (B)

     ➔ ukupno za 1 razinu: **3B + 2d koraka, O(B)** (K = 102)

     - *primjer: K = 2 * 102 = 204* $dubina = \frac{\log 1024}{\log 32} = \frac{10}{5} = 2$

     - složenost: **O(B $\log_B$ (n))**

APRAM REDUCIRANJE

- Primjer: n = 1024, B = $\sqrt{p}$ = 32, d = 3

  b) Algoritam uz **p = n/T** procesora (manji br. procesora)

  - pomoćni parametar **$T = B \cdot \log_B(n)$** ➔ "cijena" sinkronizacije u B-arnom stablu
  - APRAM **p = n/T** = 16, novi **B' = 4**
  - svaki procesor **slijedno zbraja** T elemenata:

    (d+T-1) + (T-1) + B' = 133 koraka, O(T)

    glob.R OP sinkr.
  - dobiveni niz duljine n/T reducira se **B'-arnim stablom**:
    - dubina: $\log_{B'}(n/T) = 2$
    - **jedna razina: (3*B' + 2*d) = 18 koraka**
  - **ukupno: 133 + 2*18 = 169 koraka**
  - složenost: **$O(T) = O(B \log_B(n))$** asimptotski ISTA složenost, ali manji broj koraka

ZADATAK

- Rasporedite operacije koje izvode četiri procesora na APRAM računalu u što manji broj asinkronih odsječaka odvojenih sinkronizacijskim ogradama. Varijable A, B, C i D se nalaze u globalnoj memoriji, a znak „ * “ označava lokalne operacije. Pretpostavite da prvi sa izvođenjem počinje procesor 1. Redoslijed operacija koje treba izvesti pojedini procesor treba ostati očuvan.

Procesor 1	Procesor 2	Procesor 3	Procesor 4
Čitaj A Čitaj B * Piši C	Čitaj B * * Piši B	Čitaj C Čitaj A * Piši A	Piši D * Piši D

	Procesor 1	Procesor 2	Procesor 3	Procesor 4
Odsječak 1	Čitaj A Čitaj B * Piši C			Piši D * Piši D
Odsječak 2		Čitaj B * * Piši B	Čitaj C Čitaj A * Piši A	